

Spring Boot MicroService

Why Spring Cloud?

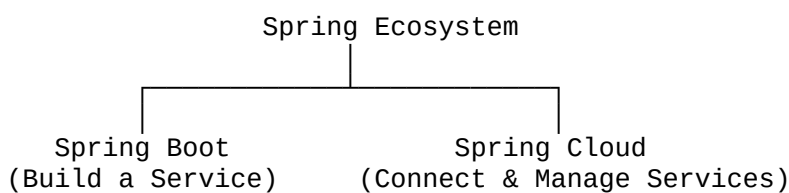
Spring Boot helps us build individual microservices quickly. However, building a production-ready microservices architecture requires solving many distributed systems problems such as service discovery, centralized configuration, API gateway, fault tolerance, distributed tracing, messaging, and load balancing.

Instead of reinventing solutions for these common challenges, Spring provides the Spring Cloud ecosystem.

Define Spring Cloud:

Spring Cloud is an umbrella of projects that extends Spring Boot with tools and integrations for building cloud-native and microservices-based applications. It provides reusable solutions to common distributed system challenges so developers can focus on business logic rather than infrastructure concerns.

visually



Example

Suppose you're building a banking system.

With **Spring Boot**, you create:

- Account Service
- Card Service
- Loan Service
- Notification Service

Now these services need to:

- Find each other → Service Discovery (Eureka or Kubernetes Service Discovery)
- Externalize configuration → Config Server
- Route requests → API Gateway
- Communicate reliably → OpenFeign, WebClient, Spring Cloud Stream
- Handle failures → Circuit Breaker (Resilience4j)
- Trace requests → OpenTelemetry, Micrometer

These concerns are where **Spring Cloud** helps.

One-line definition

Spring Cloud is a collection of projects that provides production-ready solutions for common distributed systems challenges encountered while building microservices with Spring Boot.

One important update for modern Spring (Current industry direction.)

Also mention to your participants that **Spring Cloud is an evolving ecosystem**. Some older Netflix components, such as Hystrix and Ribbon, have been retired. Today, common choices are:

- Spring Cloud Gateway
- Spring Cloud Config
- OpenFeign (or Spring Framework 7 `HttpServiceProxy/HttpExchange` where appropriate)
- Resilience4j
- Spring Cloud Stream
- Micrometer + OpenTelemetry
- Kubernetes-native service discovery in cloud environments

Netflix OSS

Spring Cloud originally integrated **Netflix OSS** projects because they solved many microservice problems before Kubernetes became mainstream. Over time, many Netflix projects were deprecated or entered maintenance mode, and Spring Cloud replaced them with more modern alternatives.

Here's a comprehensive mapping.

Microservice Concern	Netflix OSS Tool	Spring Cloud Integration	Modern Replacement
Service Discovery	Eureka	Spring Cloud Netflix Eureka	Kubernetes Service Discovery , Eureka (still supported but less common on K8s)
Client-side Load Balancing	Ribbon	Spring Cloud Netflix Ribbon	Spring Cloud LoadBalancer
Circuit Breaker	Hystrix	Spring Cloud Netflix Hystrix	Resilience4j
API Gateway	Zuul	Spring Cloud Netflix Zuul	Spring Cloud Gateway
Distributed Configuration	Archaius	Spring Cloud Config (not based on Archaius)	Spring Cloud Config , Kubernetes ConfigMaps & Secrets
Distributed Tracing	Sleuth (not Netflix)	Spring Cloud Sleuth	Micrometer Tracing + OpenTelemetry
HTTP Client	Feign	Spring Cloud OpenFeign	OpenFeign (still widely used) or Spring Framework Http Interface (@HttpExchange)
Messaging	—	Spring Cloud Stream	Spring Cloud Stream (Kafka/RabbitMQ binders)

Microservice Concern	Netflix OSS Tool	Spring Cloud Integration	Modern Replacement
Bus/Event Propagation	—	Spring Cloud Bus	Spring Cloud Bus
Distributed Session	—	Spring Session	Spring Session
Distributed Task Scheduling	—	Spring Cloud Task	Spring Cloud Task
Batch Orchestration	—	Spring Cloud Data Flow	Spring Cloud Data Flow

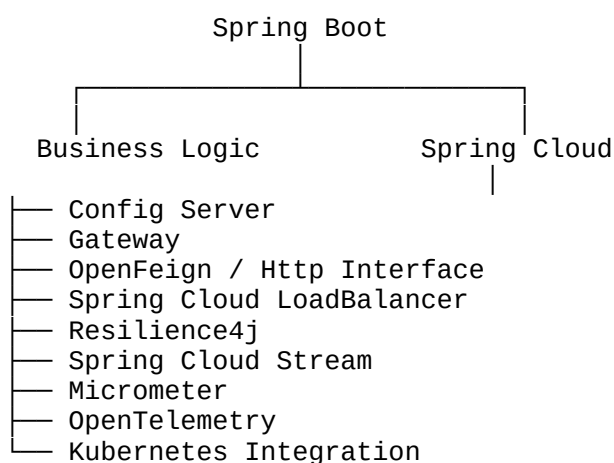
Netflix OSS ecosystem

These are the major Netflix OSS projects that became popular in Java microservices.

Netflix Project	Purpose	Status
Eureka	Service Discovery	Still supported, less common with Kubernetes
Ribbon	Client-side Load Balancer	Retired
Hystrix	Circuit Breaker	Retired
Zuul 1	API Gateway	Retired
Archaius	Distributed Configuration	Maintenance mode
Feign	Declarative REST Client	Moved to OpenFeign and still actively used
Turbine	Hystrix Dashboard Aggregator	Obsolete
Hystrix Dashboard	Circuit Breaker Dashboard	Obsolete

What a modern Spring Cloud project looks like

Instead of Netflix OSS, today's stack typically looks like:



If deploying on Kubernetes

Many Spring Cloud responsibilities are handled by Kubernetes itself.

Feature	Spring Cloud (VM/Bare Metal)	Kubernetes Equivalent
Service Discovery	Eureka	Kubernetes Services + DNS
Load Balancing	Spring Cloud LoadBalancer	Kubernetes Service
Configuration	Config Server	ConfigMap & Secret
Health Checks	Spring Boot Actuator	Liveness & Readiness Probes
Scaling	Manual/Autoscaling	Horizontal Pod Autoscaler
Self Healing	Spring Retry	Kubernetes Controller
Gateway	Spring Cloud Gateway	Ingress Controller / Gateway API

Recommend in 2026

For a modern Spring Boot microservices course, I'd focus on:

- Spring Boot
- Spring Cloud Config
- Spring Cloud Gateway
- OpenFeign (and introduce `@HttpClient` as the newer Spring Framework HTTP client abstraction)
- Spring Cloud LoadBalancer
- Resilience4j
- Spring Cloud Stream (Kafka)
- Micrometer
- OpenTelemetry
- Prometheus
- Grafana
- Docker
- Kubernetes

I would still spend one session explaining the Netflix OSS components (Eureka, Ribbon, Hystrix, Zuul) because you'll encounter them in legacy enterprise applications, but I would present them as historical context and contrast them with their modern replacements. This helps students understand both existing codebases and current best practices.